

# Sistema de Recuperação

---



Eduardo Ogasawara

[eogasawara@cefet-rj.br](mailto:eogasawara@cefet-rj.br) | <https://eic.cefet-rj.br/~eogasawara>



Sistemas de banco de dados enfrentam diversos tipos de falhas que podem comprometer a integridade dos dados. Compreender essas classificações é fundamental para implementar mecanismos de recuperação eficazes.

---

## Falha de Transação

**Erros lógicos:** A transação não pode ser concluída devido a alguma condição de erro interno, como violação de restrições ou divisão por zero.

**Erros de sistema:** O sistema de banco de dados precisa encerrar uma transação ativa devido a condições como deadlock ou timeout de recursos.

---

## Falha do Sistema

Uma queda de energia ou outra falha de hardware ou software causa a parada do sistema (crash). Opera sob a **suposição de parada segura (fail-stop)**: assume-se que o conteúdo do armazenamento não-volátil não é corrompido pela falha.

Os sistemas de banco de dados possuem numerosas verificações de integridade para prevenir corrupção dos dados em disco.

---

## Falha de Disco

Um crash de cabeça de leitura ou falha similar do disco destrói todo ou parte do armazenamento em disco. A destruição é considerada detectável através de checksums utilizados pelos drives de disco.

Considere uma transação  $T_i$  que transfere \$50 da conta  $A$  para a conta  $B$ . Esta operação requer duas atualizações: subtrair 50 de  $A$  e adicionar 50 a  $B$ . O desafio surge quando uma falha ocorre após uma modificação ser feita, mas antes que ambas sejam completadas.

---

## Dilema da Modificação

Modificar o banco de dados sem garantir que a transação será confirmada pode deixar o banco em estado inconsistente. Por outro lado, não modificar pode resultar em perda de atualizações se a falha ocorrer logo após o commit.

---

## Durante Processamento Normal

Ações tomadas durante o processamento normal das transações para garantir que informação suficiente existe para recuperação.

---

## Estrutura dos Algoritmos

Os algoritmos de recuperação possuem duas partes fundamentais que trabalham em conjunto para garantir a integridade dos dados.

---

## Após Falha

Ações tomadas após uma falha para recuperar o conteúdo do banco de dados a um estado que garanta atomicidade, consistência e durabilidade.

---

## Armazenamento Volátil

### Não sobrevive a crashes do sistema

- Memória principal (RAM)
- Memória cache
- Registradores de CPU

Dados perdidos em caso de falha de energia ou crash.

---

## Armazenamento Não-Volátil

### Sobrevive a crashes do sistema

- Discos magnéticos (HDD)
- Unidades de estado sólido (SSD)
- Fita magnética
- Memória flash
- RAM não-volátil

Ainda pode falhar, perdendo dados em algumas situações.

---

## Armazenamento Estável

### Forma mítica que sobrevive a todas as falhas

Aproximado mantendo múltiplas cópias em mídias não-voláteis distintas. Representa o ideal teórico de persistência absoluta.

Consulte o livro-texto para detalhes sobre implementação de armazenamento estável.

O armazenamento estável é aproximado na prática mantendo múltiplas cópias de cada bloco em discos separados — possivelmente em sites remotos. O objetivo é sobreviver a qualquer falha isolada.

**Falha parcial:** bloco de destino pode conter informação incorreta se a transferência for interrompida.

**Falha total:** bloco de destino nunca foi atualizado se a operação falhar completamente.

Em ambos os casos, o protocolo de escrita segura garante que pelo menos uma cópia válida sempre exista.

ENTRADA: bloco B, conjunto de cópias {disco\_1, disco\_2, ..., disco\_n}

PRÉ-CONDIÇÃO: pelo menos uma cópia válida de B existe em disco estável

PASSO 1 — Escrever na primeira cópia

escrever(B → disco\_1)

SE falha durante escrita:

disco\_1 pode conter dado corrompido ou antigo

▷ disco\_2 ainda contém cópia válida

PASSO 2 — Verificar conclusão da primeira escrita

AGUARDAR confirmação de escrita bem-sucedida em disco\_1

PASSO 3 — Escrever nas cópias restantes

PARA CADA disco\_i em {disco\_2, ..., disco\_n}:

escrever(B → disco\_i)

AGUARDAR confirmação

PASSO 4 — Confirmar saída completa

Saída considerada completa apenas após todas as escritas

serem confirmadas com sucesso

SAÍDA: bloco B persistido em todas as n cópias

PÓS-CONDIÇÃO: uma falha durante a escrita nunca corrompe todas

as cópias; os dados sobrevivem à perda de até n-1

cópias (na recuperação, cópias divergentes são

reconciliadas a partir de uma cópia válida)

---

## Blocos Físicos

São aqueles blocos que residem permanentemente no disco.

Representam o armazenamento persistente de longo prazo dos dados do banco de dados.

---

## Blocos de Buffer

São os blocos que residem temporariamente na memória principal.

Fornecem acesso rápido aos dados durante o processamento de transações.

---

## Operações de Movimentação

As movimentações de blocos entre o disco e a memória principal são iniciadas através de duas operações fundamentais:

---

### input( $B$ )

Transfere o bloco físico  $B$  para a memória principal, tornando-o disponível para leitura e modificação.

---

### output( $B$ )

Transfere o bloco de buffer  $B$  para o disco, substituindo o bloco físico apropriado. Esta operação torna as modificações persistentes.

**Simplificação assumida:** Cada item de dados cabe em um único bloco e é armazenado dentro dele. Esta suposição simplifica o gerenciamento de dados e as operações de recuperação.

Para garantir atomicidade apesar das falhas, primeiro enviamos informações descrevendo as modificações para armazenamento estável sem modificar o próprio banco de dados. Esta abordagem fundamental separa o registro das intenções da execução real das mudanças.

## Mecanismos Baseados em Log

Estudamos **mecanismos de recuperação baseados em log** em detalhes. Esta é a abordagem mais amplamente utilizada na indústria devido à sua eficiência e robustez.

---

### Conceitos-Chave

Primeiro apresentamos os conceitos fundamentais que sustentam a recuperação baseada em log.

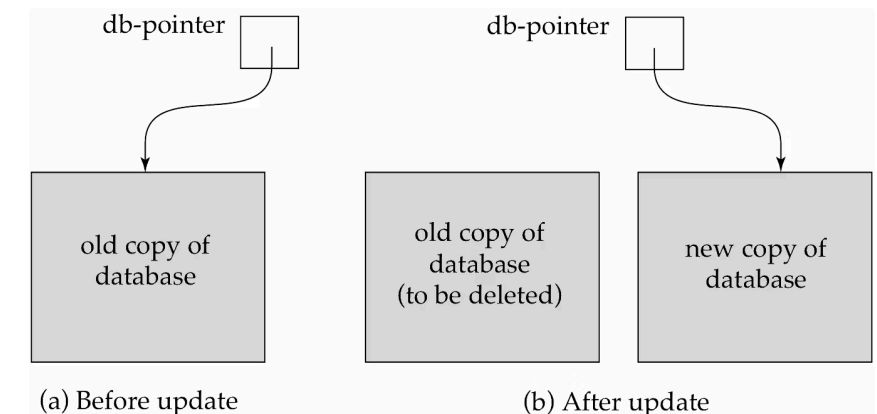
---

### Algoritmo de Recuperação

Então apresentamos o algoritmo real de recuperação com todos os detalhes de implementação.

## Alternativas

📄 **Shadow-copy e Shadow-paging:**  
Alternativa menos utilizada para recuperação.



*Exemplo de shadow-copy*

Um **log** é uma sequência de **registros de log** que mantém informações sobre atividades de atualização no banco de dados. O log é mantido em armazenamento estável para garantir sua persistência mesmo em caso de falhas.

---

#### <Ti start>

Quando a transação *Ti* inicia, ela se registra escrevendo um registro de log de início.

#### <Ti, X, V1, V2>

Antes de *Ti* executar `write(X)`, um registro de log é escrito, onde *V1* é o **valor antigo** de *X* antes da escrita, e *V2* é o **valor novo** a ser escrito em *X*.

#### <Ti commit>

Quando *Ti* termina sua última instrução, o registro de log de commit é escrito, marcando o sucesso da transação.

---

## Duas Abordagens Utilizando Logs

### Modificação Imediata do Banco de Dados

Permite atualizações antes do commit da transação.

### Modificação Diferida do Banco de Dados

Adia atualizações até o commit da transação.

O esquema de **modificação imediata** permite que atualizações de uma transação não confirmada sejam feitas no buffer, ou no próprio disco, antes que a transação seja confirmada. Esta abordagem oferece flexibilidade, mas requer cuidados especiais para garantir a recuperabilidade.

---

### Regra de Escrita Antecipada do Log (WAL)

O registro de log de atualização **deve ser escrito antes** que o item do banco de dados seja escrito. Assumimos que o registro de log é enviado diretamente para armazenamento estável.

*Nota:* Veremos posteriormente como adiar a saída de registros de log até certo ponto para melhorar o desempenho.

---

### Flexibilidade de Saída

A saída de blocos atualizados para o disco pode ocorrer a qualquer momento antes ou após o commit da transação. A ordem em que os blocos são enviados pode ser diferente da ordem em que foram escritos.

---

## Esquema de Modificação Diferida

O esquema de **modificação diferida** realiza atualizações no buffer/disco somente no momento do commit da transação.

- **Vantagem:** Simplifica alguns aspectos da recuperação, pois transações não confirmadas não afetam o banco de dados
- **Desvantagem:** Tem o overhead de armazenar cópia local das modificações até o commit

Uma transação é considerada confirmada (committed) quando seu registro de log de commit é enviado para armazenamento estável. Este é o ponto crítico que determina se as modificações da transação se tornam permanentes e visíveis para outras transações.

---

### Pré-requisito para Commit

Todos os registros de log anteriores da transação **devem ter sido enviados** para armazenamento estável antes que o registro de commit possa ser escrito. Esta ordem garante a recuperabilidade completa.

---

### Escritas Pendentes no Buffer

As escritas realizadas por uma transação podem ainda estar no buffer quando a transação confirma, e podem ser enviadas para o disco posteriormente. O importante é que o log garanta que essas escritas possam ser refeitas se necessário.

 **Ponto-chave:** A durabilidade não depende de todas as modificações estarem fisicamente no disco no momento do commit, mas sim de todos os registros de log estarem em armazenamento estável. Isso permite otimizações significativas de desempenho.

Com transações concorrentes, todas as transações compartilham um único buffer de disco e um único log. Um bloco de buffer pode ter itens de dados atualizados por uma ou mais transações, criando desafios únicos para o sistema de recuperação.

Assumimos que se uma transação  $T_i$  modificou um item, nenhuma outra transação pode modificar o mesmo item até que  $T_i$  tenha confirmado ou abortado.

Em outras palavras, as atualizações de transações não confirmadas não devem ser visíveis para outras transações.

---

## Por Que Esta Restrição?

**Problema do cascadeamento:** Como realizar undo se  $T_1$  atualiza A, depois  $T_2$  atualiza A e confirma, e finalmente  $T_1$  precisa abortar? Seria necessário desfazer  $T_2$ , causando um efeito cascata indesejável.

---

## Implementação Prática

Pode ser garantido obtendo **locks exclusivos** em itens atualizados e mantendo os locks até o final da transação. Esta é a abordagem de **bloqueio estrito em duas fases (strict two-phase locking)**.

---

## Intercalação no Log

Os registros de log de diferentes transações podem ser intercalados no log, refletindo a execução concorrente. O sistema de recuperação deve ser capaz de processar essa intercalação corretamente.

As operações **undo** e **redo** são as duas operações centrais usadas durante a recuperação para restaurar a consistência do banco de dados após falhas.

**undo(Ti):** Restaura todos os itens de dados atualizados por *Ti* para seus valores antigos, percorrendo o log de trás para frente. Escreve um compensation log record `<Ti, X, V>` para cada item restaurado e, ao final, escreve `<Ti abort>`.

**redo(Ti):** Define todos os itens de dados atualizados por *Ti* para seus valores novos, percorrendo o log para frente. Nenhum registro de log é escrito. As operações são idempotentes.

*Ambas as operações são idempotentes — podem ser repetidas com segurança.*

ENTRADA: **log** (sequência de registros), *Ti* (transação alvo)

PRÉ-CONDIÇÃO: **log** em armazenamento estável; *Ti* aparece **no log**

PASSO 1 — **undo(Ti)**

Percorrer o **log** de **trás** para frente a partir **do** último registro de *Ti*

PARA CADA registro `<Ti, X, V_antigo, V_novo>` encontrado:

escrever(*X*, *V\_antigo*)      ▷ restaura valor antigo

append\_log(`<Ti, X, V_antigo>`) ▷ compensation **log** record

Ao terminar:

append\_log(`<Ti abort>`)

PASSO 2 — **redo(Ti)**

Percorrer o **log** de frente para **trás** a partir **do** primeiro registro de *Ti*

PARA CADA registro `<Ti, X, V_antigo, V_novo>` encontrado:

escrever(*X*, *V\_novo*)      ▷ reaplica valor novo

▷ Nenhum registro de **log** é escrito durante o **redo**

SAÍDA: banco de dados com *Ti* desfeita (undo) ou refeita (**redo**)

PÓS-CONDIÇÃO: estado consistente; operações são idempotentes

Sem checkpointing, a recuperação precisa varrer o log inteiro — um processo lento que pode envolver horas ou dias de registros. Além disso, transações que já persistiram suas atualizações no disco seriam desnecessariamente refeitas. O checkpointing cria um ponto seguro e conhecido no log, limitando o quanto a recuperação precisa retroceder.

- ❏ Após um checkpoint, apenas transações ativas em <checkpoint L> ou iniciadas depois precisam ser consideradas na recuperação.

*Tradeoff: o checkpointing pausa todas as atualizações momentaneamente para garantir consistência global.*

ENTRADA: log, conjunto de transações ativas  $T_{ativas}$

PRÉ-CONDIÇÃO: sistema em operação normal; log em armazenamento estável

PASSO 1 — Pausar atualizações

Suspender todas as atualizações em andamento

▷ nenhuma modificação ocorre durante o checkpoint

PASSO 2 — Flush do log

Enviar todos os registros de log da memória principal para armazenamento estável

PASSO 3 — Flush dos buffers modificados

Enviar todos os blocos de buffer modificados para o disco

▷ garante que modificações pendentes sejam persistidas

PASSO 4 — Escrever registro de checkpoint

append\_log(<checkpoint L>)

forçar para armazenamento estável

onde L = lista de todas as transações ativas no momento

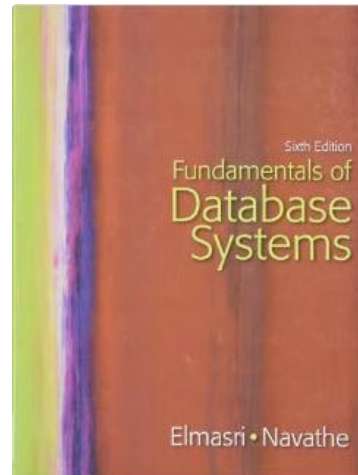
PASSO 5 — Retomar atualizações

Liberar as transações suspensas para continuar

SAÍDA: registro <checkpoint L> em armazenamento estável

PÓS-CONDIÇÃO: recuperação futura só precisa examinar o log a partir do checkpoint mais recente

## Referências

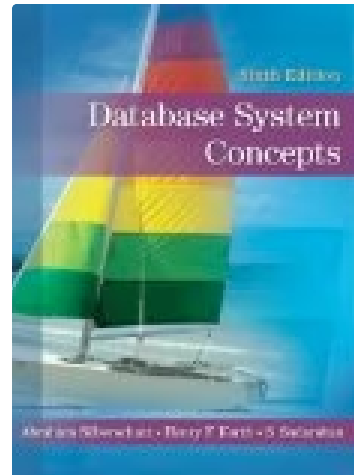


---

**Elmasri & Navathe**

**Fundamentals of Database Systems**

Pearson, 2016

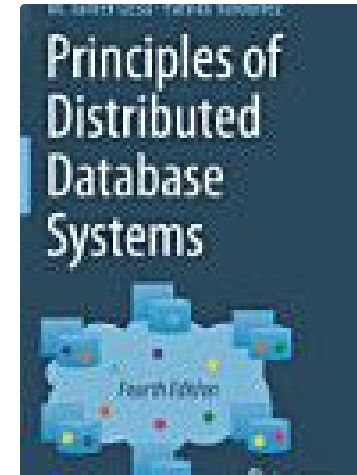


---

**Korth, Sudarshan & Silberschatz**

**Database System Concepts**

McGraw-Hill, 2019



---

**Özsu & Valduriez**

**Principles of Distributed Database Systems**

Springer Nature, 2019